

ENGINEERING REPORT / SOFTWARE RELEASE 0.1.0

NVIDIA Industrial Catalog Extraction Pipeline

How it works, system architecture, evidence controls, validation results, and the path to full-corpus production

Prepared for the Industrial Data Catalogs program

Status as of 7 August 2026 • Local + NVIDIA server implementation

8 × A100 80 GB
Verified NVIDIA GPU server

220 PDFs
Transferred and inventory-verified

17,837 pages
Two pinned corpus profiles

140+ tests passed
Local and remote verification

Status boundary

The architecture and bounded production validation are complete. The full 17,837-page corpus has not yet been processed; manifest-pinned page-window sharding and deterministic merge controls are required before that claim is valid.

Public repository

Repository-relative code, tests, documentation, and report artifacts

REPORT MAP

Contents and reading guide

Sections	Theme	What the reader gets
1–3	Decision context	Executive result, design goals, infrastructure, and corpus scope
4–7	Architecture	Routing, VLM extraction, structured parsing, evidence guards, and resilience
8–10	Data and operations	Canonical schema, two-tier RAG, checkpoints, deployment, and monitoring
11–12	Proof	Validation runs, incident replay, integrity results, and throughput interpretation
13–14	Production path	Risks, manifest sharding, deterministic merge, and delivery roadmap
Appendices	Reference	Implementation map, output artifacts, glossary, and authoritative model sources

Fast path for decision-makers

Read the Executive Summary, Verified Results, Risk Register, and Production Scale Roadmap. The remaining sections provide implementation-level traceability.

Independent research and licensing boundary

This is independent research software and is not an NVIDIA product or endorsement. Repository code is offered under Apache-2.0; that license does not grant rights to NVIDIA model weights, third-party dependencies, or source catalog content. Review each upstream model, dependency, and data license before use or redistribution.

DECISION BRIEF

1. Executive Summary

The NVIDIA Industrial Catalog Extraction Pipeline is an evidence-preserving system for converting heterogeneous industrial PDFs into grounded product records and a citation-bearing retrieval knowledge base. Its defining rule is that model output does not become canonical merely because inference succeeded: every field must resolve to page evidence, survive deterministic source guards, and pass typed validation before persistence.

The system uses three NVIDIA model services on dedicated A100 GPUs. NVIDIA Nemotron Parse v1.2 reads complex document pages; Llama 3.1 Nemotron Nano 8B v1 converts evidence into schema-constrained product JSON; and Nemotron 3 Embed 8B BF16 creates retrieval vectors. Native PDF extraction handles clean digital pages without paying VLM cost, while an optional OCR verification hook can add independent identifier evidence when deployed.

What is implemented and proven

- End-to-end code, deployment scripts, service health checks, deterministic routing, extraction, guarded parsing, SQLite persistence, two-tier RAG, monitoring, checkpoints, and audit artifacts are implemented.

- The verified server exposes eight NVIDIA A100-SXM4 80 GB GPUs, approximately 2 TB of system memory, 255 logical CPUs, and roughly 17.4 TB free on the data volume at inspection time.
- A canonical transfer containing 220 PDFs and 17,837 pages was hash-verified on the server. Vendor catalogs and the broader reference library are treated as separate campaigns because their semantics differ.
- A 20-document production validation traversed every stage; a hardened two-document replay then resolved both observed extraction failures while keeping unsupported products out of the catalog.
- Local and remote verification completed with more than 140 tests passing and Ruff reporting no lint findings.

What is not yet complete

The complete 17,837-page campaign has not been executed. The current benchmark wrapper is intentionally bounded and the runner is sequential. Whole-document knowledge-base replacement also prevents safe page-window sharding today. A manifest-pinned campaign runner, isolated work-unit outputs, and deterministic merge must be implemented before full-corpus completion can be claimed.

Recommended decision

Approve the manifest/sharding/merge work as the next production milestone, then run a two-worker vendor pilot. Expand to a third worker only when aggregate throughput improves without rising queue time, invalid JSON, timeouts, or GPU pressure.

PURPOSE AND BOUNDARIES

2. Objectives, Scope, and Design Principles

2.1 Business objective

Industrial catalogs contain high-value product identity and engineering data, but layouts vary by vendor, edition, page type, and scan quality. The pipeline's purpose is to turn those files into searchable, source-grounded records that can support product discovery, specification lookup, comparison, and downstream retrieval-augmented generation.

Target output	Examples	Admission requirement
Product identity	Manufacturer, product/part name, part number, series, family	Exact or source-justified evidence and deterministic manufacturer/cardinality rules
Specifications	Voltage, dimensions, material, rating, tolerance, capacity	Typed value plus source evidence; no unsupported normalization
Operating details	Conditions, compatibility, compliance, application notes	Validated field with document/page/element citation
Other details	Description, ordering context, features, warnings	Schema-valid content traceable to extracted evidence
Retrieval evidence	Page element, exact excerpt, bbox, score	Indexed independently of canonical product admission

2.2 Architectural principles

Principle	How the implementation expresses it
Evidence first	Keep exact source text, document identity, page, bounding box, extraction method, provider payload, and model metadata.
Deterministic around probabilistic	Use explicit thresholds, stable identifiers, strict schemas, source guards, typed validation, and idempotent replacement.
Fail closed	Do not convert blank/unsupported model output or unresolved nonblank pages into canonical facts.
Selective acceleration	Use native PDF evidence when it is good; reserve the document VLM for scans, tables, columns, and visually complex layouts.
Two-tier retrieval	Keep raw evidence searchable even when a candidate does not qualify for the canonical product catalog.
Operational boundedness	Limit candidate pages, wall time, GPUs, and output directories until campaign-scale correctness is implemented.

VERIFIED PLATFORM

3. NVIDIA Infrastructure, Model Stack, and Corpus

3.1 Server capacity

Resource	Verified capacity	Pipeline use
GPU	8 × NVIDIA A100-SXM4 80 GB (640 GB aggregate VRAM)	Three isolated inference services; two GPUs reserved; three free for measured expansion
System memory	Approximately 2 TB RAM	Large page buffers, concurrent workers, model/service overhead, and merge operations
CPU	255 logical CPUs	PDF parsing, image rendering, orchestration, validation, and SQLite work
Data volume	About 20.4 TB total / 17.4 TB free at inspection	Corpus, checkpoints, run artifacts, databases, vectors, and published releases

3.2 NVIDIA model responsibilities

Stage	Model / endpoint	GPU	Role and operating contract
Document VLM	nvidia/NVIDIA-Nemotron- Parse-v1.2 127.0.0.1:8001	1	Visual document parsing: reading order, text, tables, element classes, and normalized bounding boxes; max sequences 8
Structured LLM	nvidia/Llama-3.1-Nemotron- Nano-8B-v1 127.0.0.1:8003	3	Guided JSON for product identity/specifications; source hints and post-response guard; max sequences 16

Stage	Model / endpoint	GPU	Role and operating contract
Embeddings	nvidia/Nemotron-3-Embed-8B-BF16 127.0.0.1:8004	5	Passage/query embeddings for exact cosine retrieval; production output documented as 4,096 dimensions; max sequences 16

The structured LLM runs through vLLM 0.20.0 with the xgrammar structured-output backend and disabled arbitrary whitespace. GPU 0 and GPU 2 belong to pre-existing workloads and are outside project control. GPUs 4, 6, and 7 remain unassigned until a measured capacity plan authorizes them.

3.3 Corpus inventory and campaign separation

Corpus	PDFs	Pages	Intended extraction profile
Vendor product catalogs	110	6,069	Manufacturer, part identity, specifications, operating details
Industrial reference library	110	11,768	Reference-document extraction with its own quality policy
Combined transfer	220	17,837	Two pinned campaigns; optionally merged into one retrieval release

Coverage caveat

Nineteen vendor PDFs exceed 50 pages and contain 5,247 pages. A 50-page-per-document cap covers only 1,772 of 6,069 vendor pages (29.2%); raising only the document limit cannot create full coverage.

The research corpus was transferred through a hash-verified 918,048,768-byte archive. Neither that archive nor the source PDFs are distributed with the public repository. The exact deployment code and data identities were retained in the private operational record.

SYSTEM STRUCTURE

4. End-to-End Architecture

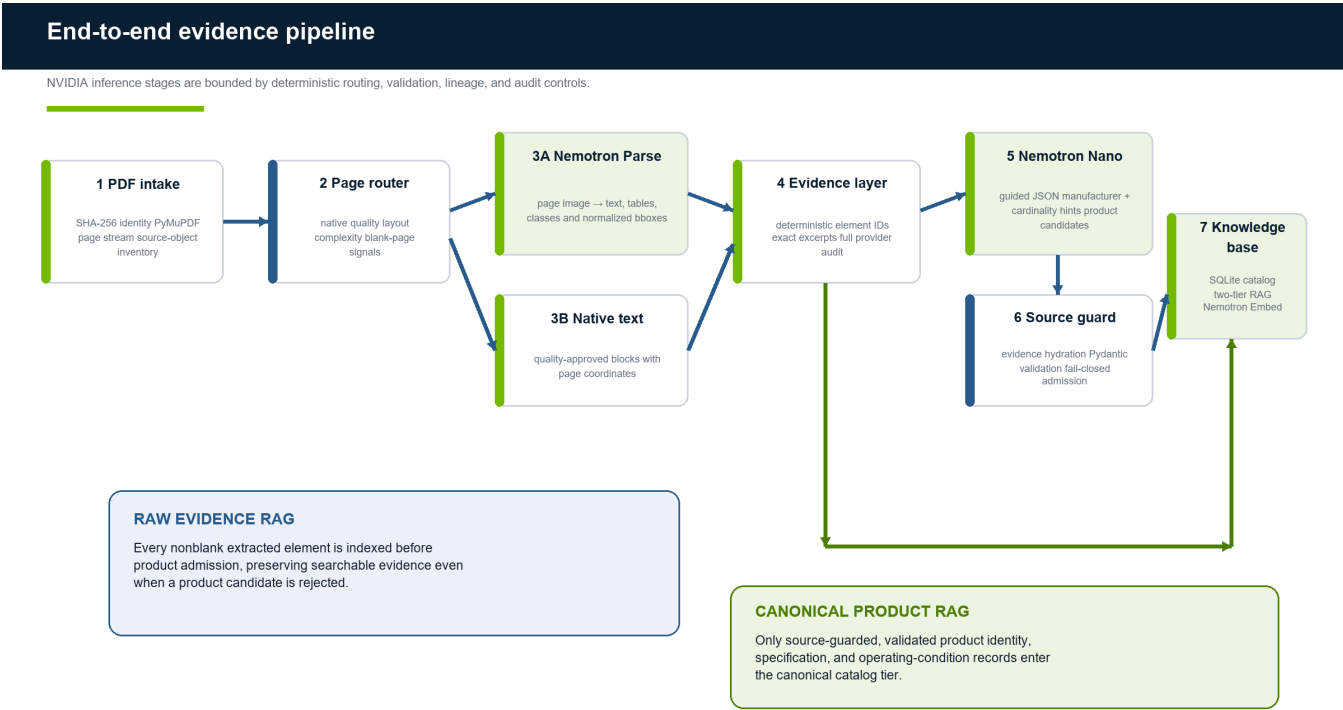


Figure 1. End-to-end evidence and knowledge-base architecture.

4.1 Component responsibilities

Component	Input	Output	Control boundary
Document intake	PDF file	Document SHA, page iterator, metadata	Content-addressed identity; lazy page rendering
Page router	Native signals + layout inventory	native_text / document_vlm / OCR flags + reasons	Fixed thresholds and ordered reason codes
Nemotron Parse	Rendered page image + task prompt	Text/table/element evidence	Bounded retries; semantic response validation
Evidence layer	Native/VLM/OCR elements	Canonical elements with IDs and citations	No model-generated field is evidence by itself
Nemotron Nano	Bounded evidence chunk + guided schema	Product candidates	Manufacturer and cardinality hints constrain output
Guard + validator	Candidates + canonical evidence	Accepted, rejected, or invalid records	Cross-document and unsupported claims fail closed
Catalog + RAG	Accepted products + all page elements	SQLite/JSONL + vectors/citations	Canonical tier separated from raw evidence tier

EXTRACTION MECHANICS

5. How the Pipeline Works

5.1 Document identity and page intake

1. Hash the PDF and assign the stable document ID sha256:<file hash>.
2. Stream pages in one-based order with PyMuPDF; do not render every page eagerly.
3. Inventory native text/blocks, page dimensions, tables, multi-column structure, image coverage, drawings, embedded images, annotations, and links.
4. Render PNG bytes only when the selected VLM/OCR path requires pixels.

This division keeps large digital catalogs inexpensive while retaining the source-object detail needed to distinguish truly blank pages from nontext visual content.

5.2 Deterministic page routing

Signal	Default threshold / trigger	Interpretation
Non-whitespace characters	≥ 80	Minimum native text volume
Words	≥ 12	Rejects thin headers and fragments
Printable ratio	≥ 0.95	Avoids corrupt encoding
Alphanumeric ratio	≥ 0.35	Requires meaningful text content
Replacement-character ratio	≤ 0.01	Limits decoding damage
Layout trigger	Table, multi-column, or image coverage ≥ 0.45	Routes page to the document VLM despite usable native text

The routing decision stores every measured signal and an ordered reason-code list. A page can therefore be reproduced and explained without asking the model why it was selected.

5.3 NVIDIA Nemotron Parse extraction

VLM-routed pages are rendered and submitted as base64 image URLs to the local OpenAI-compatible endpoint. Parse v1.2 receives the task prompt shown below; sampling is deterministic (temperature 0, top_k 1) and output length is bounded to 8,192 tokens.

```
</s><s><predict_bbox><predict_classes><output_markdown>
<predict_no_text_in_pic>
temperature=0 top_k=1 max_tokens=8192 repetition_penalty=1.1
```

Normalization accepts JSON element payloads, Nemotron's tagged class/text/bounding-box format, or a final document-text element when a supported response cannot be split further. Tagged Parse bounding boxes are explicitly labeled as normalized coordinates; native blocks retain PyMuPDF page coordinates.

5.4 Optional OCR verification

OCR is implemented as an independent verifier, not a destructive replacement. When an endpoint is configured, VLM pages can be OCR-checked and native pages containing identifier-like tokens can receive an additional evidence channel. The current production example does not deploy an OCR model, so the hook is present but inactive.

5.5 Grounded evidence elements

Field group	Preserved content
Source identity	Document ID, source path, one-based page, sequence number
Evidence	Exact text, element type, bounding box, confidence
Provenance	Extraction method, model identity, provider payload, metadata
Determinism	Element ID hashed from document/page/sequence/type/text/method/bbox/model
Audit	Route, reason codes, attempts, failures, fallback/blank events

STRUCTURED INTELLIGENCE

6. Product Parsing, Source Guard, and Validation

6.1 Pre-model source hints

Before Nemotron Nano is called, deterministic code examines the evidence for manufacturer candidates and product cardinality. Manufacturer hints come from legal names, labels, copyright lines, and brand evidence. Cardinality comes from explicit orderable rows rather than the LLM's preference to split or combine products.

Source condition	Guided-schema effect	Post-response enforcement
Explicit manufacturer evidence	Manufacturer and evidence IDs constrained to source-backed values	Unrecognized manufacturer/evidence pairs are rejected
Ordinary prose	At most one product candidate	Extra product objects are guard-rejected
Distinct orderable table rows	One identity-only product per exact identifier	Identifiers and row cardinality must match evidence
No justified part-number structure	Part-number extraction disabled	Invented or inferred numbers are rejected

6.2 Guided JSON and bounded regeneration

Nemotron Nano receives a strict JSON Schema through OpenAI `response_format`. Malformed or truncated JSON can trigger one bounded regeneration request. Attempts, finish reasons, provider responses, decoded content source, and error classification remain available for audit.

Structured wire contract	Implemented bound / behavior
Top level	Exactly one object with a required <code>products</code> array; unknown properties rejected

Structured wire contract	Implemented bound / behavior
Product cardinality	Maximum 32 per parse batch before stricter source-derived bounds
Required identity	Manufacturer and part name; part number optional and disabled on single-family pages
Specifications	Name and value required; optional unit; maximum 48
Other details	Fixed name/value object array; maximum 24
Evidence references	One to eight supplied element IDs per emitted value
Raw string	Nonblank and at most 2,048 characters

The wire object is intentionally smaller than the canonical product model. Trusted application code adds source identity, canonical evidence, normalized values, confidence, IDs, extraction metadata, and review status. Canonical materials, certifications, operating conditions, compatible parts, and aliases exist for future enrichment but are not currently emitted by the guided prompt.

6.3 Versioned contracts

Contract	Version
Extraction pipeline	1.1
Canonical ProductRecord / ProductBatch	1.0
Catalog SQLite schema	2
Knowledge-base schema	1.1
Source guard	source-guard-v3
Identity-row repair	identity-row-repair-v2

6.4 Evidence hydration and typed validation

Compact evidence references returned by the LLM are replaced with canonical extraction elements. Validation rejects unknown element IDs, cross-document evidence, mismatched claimed text or page values, missing required evidence, malformed fields, and unsupported normalizations. Valid siblings may still be stored when another candidate in the same ingestion batch is rejected.

Canonical admission rule

A plausible value without traceable evidence is not a product fact. It may remain searchable in raw page evidence, but it cannot enter the canonical product catalog.

Validation outcome	Representative conditions	Result
Always error	Schema failure; unknown/cross-document evidence; no part name or number; multiline/replacement-character part number	Candidate rejected

Validation outcome	Representative conditions	Result
Warning by default	Missing manufacturer, part number, specifications, or field evidence	Accepted as needs_review; becomes error in strict mode
Warning	Confidence below 0.65; duplicate/conflicting specifications; blank value; evidence-claim mismatch	Accepted with audit warning when no error exists
Accepted	Canonical model exists and no error-level issue remains	Persisted and eligible for canonical RAG
Review document	At least one accepted product carries a warning or quality issue	Terminal review status, kept separate from failure

RESILIENCE AND SAFETY

7. Retry, Blank-Page, Fallback, and Failure Policies

Page extraction decision and recovery

Transport success is not semantic success; blank, fallback, and failure outcomes are independently audited.

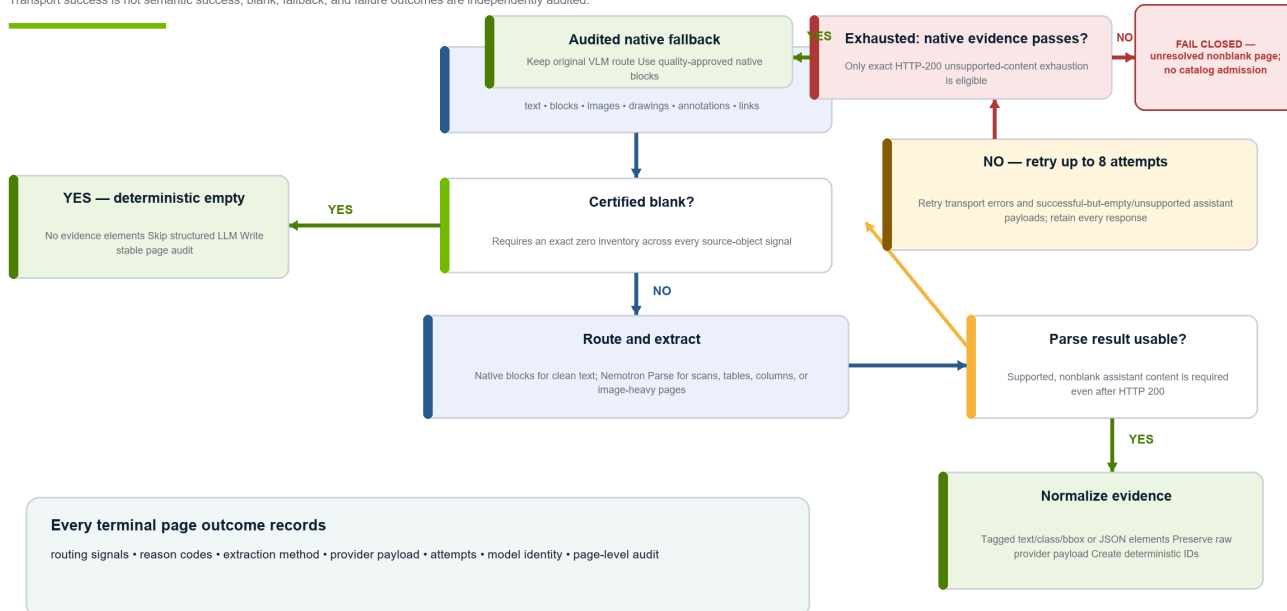


Figure 2. Page extraction decision and bounded recovery policy.

7.1 Transport and semantic retry

The shared client uses deterministic exponential retry without jitter. It retries transport failures and HTTP 408, 409, 425, 429, 500, 502, 503, and 504. The Parse configuration specifies seven retries in addition to the first request, permitting eight total attempts.

A 2xx response is not automatically accepted. The client searches supported assistant payload locations—message content, parsed objects, content blocks, tool/function arguments, completion text, reasoning-content fallbacks, and Responses-style output. Blank or unsupported successful responses are audited and retried.

7.2 Certified blank pages

Blankness is positively certified only when the source-object inventory is complete and native text, native blocks, tables, image coverage, preloaded image bytes, drawings, images, annotations, links, and multi-column structure all indicate zero/false. Missing or invalid signals never count as blank. Certified blank pages emit no element, write one stable audit, and permit the structured LLM to be skipped.

7.3 Native fallback after semantic exhaustion

Fallback is permitted only for status 200 with failure kind `unsupported_assistant_content`, at least one successful-but-invalid HTTP response, and independently quality-approved native text. The original route stays `document_vlm` and the full failure record is retained. Transport errors, server failures, mixed error sequences, and nonblank pages without meaningful native text fail closed.

Observed condition	Terminal action	Why
Exact zero source-object inventory	Certified empty; skip structured parse	Avoids unnecessary inference without hiding content
HTTP 200, supported content	Normalize to evidence	Semantic contract satisfied
HTTP 200, blank/unsupported content	Retry; retain raw response	Transport success did not produce evidence
Semantic exhaustion + good native text	Audited native fallback	Independent evidence is available
Nonblank page without usable native evidence	Fail closed	No safe source-grounded substitute
Transport/server/mixed retry exhaustion	Fail closed	Does not meet the narrow fallback predicate

CANONICAL DATA AND RETRIEVAL

8. Storage, Data Model, and Two-Tier RAG

8.1 Canonical storage model

Entity / artifact	Purpose	Lineage or integrity behavior
documents	Document status and identity	Content-addressed by SHA; review remains distinct from failure
extracted elements	Page-grounded evidence	Stable element IDs; exact text, bbox, method, model, audit
ingestion batches	One structured parse/validation decision unit	Stores raw extraction, raw LLM output, accepted/rejected counts
products	Canonical product identity and details	Only source-guarded and validated records
specifications / conditions	Typed engineering facts	Evidence links retained at field level
knowledge_chunks	Retrieval text and embedding	Deterministic content-derived chunk IDs and owner-scoped replacement

Entity / artifact	Purpose	Lineage or integrity behavior
citations	Exact retrieval provenance	Document/path/page/element/bbox/excerpt/confidence/field paths

8.2 Two retrieval tiers

Tier	Chunking	Admission rule	Primary use
Raw page evidence	One chunk per nonblank extracted page element	Indexed before product admission	Find exact page facts, rejected candidates, and unresolved content
Canonical product	Identity plus one chunk per specification/operating condition	Only validated persisted products	Reliable product search and grounded downstream RAG

Indexed passages are prefixed with `passage:` and queries with `query:`. The current store keeps float vectors as JSON in SQLite and computes exact cosine scores over all compatible vectors. Text-only mode performs escaped case-insensitive substring matching with deterministic structural ordering.

The library API can restrict retrieval by chunk tier, but the current CLI searches product and raw page-evidence tiers together. A user-facing tier selector and tier-aware ranking policy should be added before the interface is presented as a production search product.

Retrieval qualification

The production embedding model is documented as 4,096-dimensional, but current runtime validation checks only nonempty, finite, batch-consistent vectors. Model and exact dimension should become enforced release invariants before a large merged knowledge base is published.

8.3 Determinism and replacement

Stable JSON serialization and SHA-256 over text, citations, metadata, type, ordinal, and owner create content-derived chunk IDs. Embedding completes before replacement. Product, page, and document scopes are deleted and reinserted inside SQLite transactions, making retries idempotent at each owner scope. A multi-product indexing call, however, commits once per product rather than as one collection-wide transaction.

RECOVERY AND EVIDENCE CHAIN

9. Checkpointing, Auditability, and Output Artifacts

9.1 Page checkpoints

Checkpoint fingerprints include pipeline version 1.1, document/page identity, native text and block structure, dimensions, source-object metadata, preloaded image hash when present, complete routing decision, and credential-free Parse/OCR client configuration. Checkpoints are written through a temporary file, flushed, fsync'd, and atomically replaced. Restore requires matching fingerprint, document ID, and page.

Known invalidation gap

Render DPI/profile is not an explicit fingerprint field for lazy page images. The source-PDF hash protects document identity, but a rendering-profile change should directly invalidate VLM page checkpoints in the campaign runner.

9.2 Run artifacts

Artifact	Contents	Use
summary.json	Run identity, elapsed time, document/page/product/chunk counts	Operator reconciliation and release gating
documents.jsonl	Per-document terminal status and failure/review context	Audit, retry selection, and status reporting
extraction/	Page evidence, routing, provider payloads, checkpoints	Reproduction and page-level investigation
structured/	Guided JSON attempts and failure payloads	Decode/finish-reason analysis and bounded replay
catalog.sqlite3	Canonical records, evidence, ingestion batches, rejections	Transactional product system of record
knowledge_base.sqlite3	Chunks, vectors, citations	Citation-bearing retrieval
monitor.latest.json / history	GPU, endpoint, data volume, job state	Live watch and post-run proof

9.3 Audit chain

A reviewer can move from a retrieved chunk to its citation, from citation to exact element and page, from element to routing/extraction method, and from model-derived product field back through source guard, validation, provider response, and ingestion batch. Rejected products remain journaled; they are not silently discarded.

DEPLOYMENT AND OPERATIONS

10. NVIDIA Service Topology and Run Operations

Verified NVIDIA server deployment

Three isolated OpenAI-compatible inference endpoints support a sequential, bounded runner.

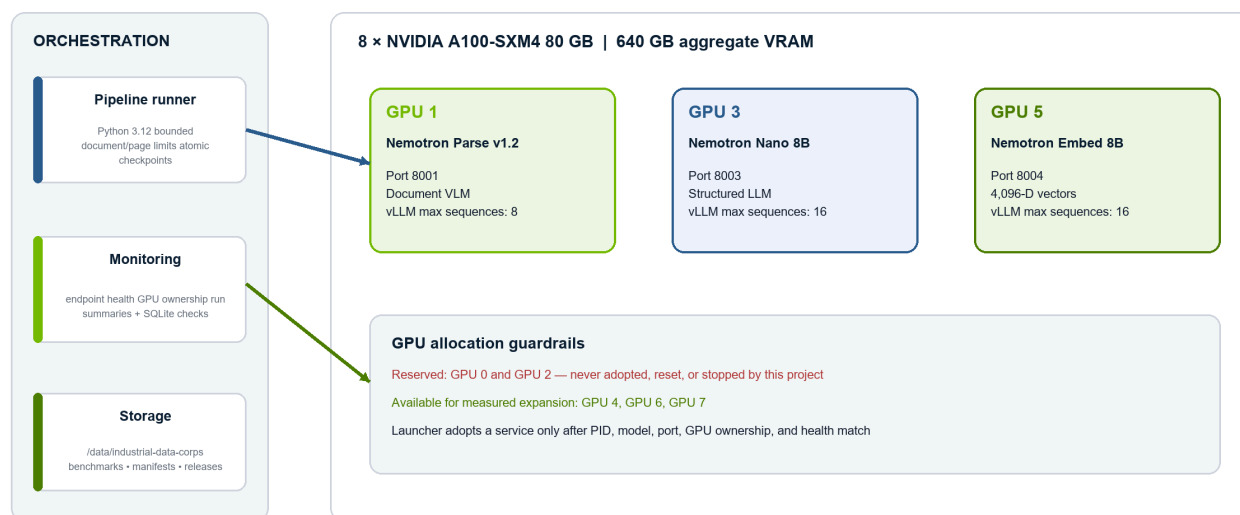


Figure 3. Verified GPU service assignment and project guardrails.

10.1 Deployment locations

Location	Path	Purpose
Repository checkout	<code>\${PROJECT_ROOT}</code>	Code, tests, configuration, scripts, and this report
NVIDIA server project	<code>\${SERVER_PROJECT_ROOT}</code>	Deployed code on the NVIDIA server
Remote data root	<code>/data/industrial-data-corps</code>	Corpus, manifests, environments, runs, status, and future releases

10.2 Service lifecycle

```
bash scripts/bootstrap_remote.sh
bash scripts/start_model_services.sh --with-embedding
curl -fsS http://127.0.0.1:8001/health
curl -fsS http://127.0.0.1:8003/health
curl -fsS http://127.0.0.1:8004/health
bash scripts/stop_model_services.sh # scoped project shutdown only
```

The launcher verifies PID, model, port, physical GPU, health, and advertised model identity before adopting a live service. Operational policy prohibits broad process kills or GPU resets because GPUs 0 and 2 carry unrelated workloads.

10.3 Bounded validation envelope

`scripts/run_benchmark.sh` defaults to 20 documents, 12 pages per document, at most 240 candidate pages, and a two-hour wall timeout. It enforces 1–100 documents, 1–50 pages per document, a 500-page maximum per invocation, 60–86,400 seconds wall time, approved physical GPU visibility, and one unique run directory.

```
INDUSTRIAL_BENCHMARK_MAX_DOCUMENTS=20 \
INDUSTRIAL_BENCHMARK_MAX_PAGES_PER_DOCUMENT=12 \
INDUSTRIAL_BENCHMARK_TIMEOUT_SECONDS=7200 \
bash scripts/run_benchmark.sh --live --pages-per-parse 1
```

10.4 Monitoring and stop conditions

- Monitor health and model identity on all three endpoints; confirm only GPUs 1, 3, and 5 are owned by the project services.
- Stop or pause on endpoint degradation, repeated invalid/truncated JSON, increasing HTTP retries/timeouts, OOM or site temperature threshold, reserved-GPU ownership, SQLite integrity/locking errors, or a data volume above 90% use.
- Keep review separate from failure. A review record contains accepted products with warnings; it is not an execution failure.
- Never share an output directory between processes. JSONL writers are process-local and document-level evidence replacement can overwrite concurrent work.

MEASURED PROOF

11. Validation Results and Incident Replay

11.1 Production validation batch

Run 20260807T064527Z-971750 exercised the complete pipeline on a bounded selection. It demonstrates integration and provides capacity evidence; it does not represent corpus completion.

Metric	Value	Interpretation
Documents attempted	20	Bounded validation selection
Review	18	Accepted with quality warnings; not failed
Failed	2	Both later resolved by hardened replay
Pages counted	105	Pages in successful/review documents
Extracted elements	1,622	Grounded page evidence
Products parsed	73	Structured candidates before all controls
Guard-rejected	41	Unsupported source/cardinality claims blocked
Products stored	31	Canonical accepted records
Invalid	1	Failed typed/evidence validation
Knowledge chunks	1,655	Raw + canonical retrieval material
Elapsed	828.999771 s	Includes failed-document time

11.2 Hardened replay of the two failures

Run 20260807T073659Z-1185130 processed one vendor catalog and one reference document after resilience hardening. It completed 24 pages in 85.468255 seconds with two review documents, zero failures, 531 extracted elements, and 531 page-evidence chunks/citations.

Case	Observed condition	Correct terminal behavior
Vendor catalog page 6	Eight successful HTTP responses contained no supported assistant content	Used 25 quality-approved native blocks; retained original VLM route and every response/fallback audit
Reference document page 2	Complete page inventory proved no text, blocks, tables, images, drawings, annotations, or links	Certified blank; emitted no element; skipped structured LLM; created no ingestion batch
Replay candidate	One structured product candidate did not satisfy source guard	Rejected deterministically; zero unsupported products entered the catalog

The replay produced 23 ingestion batches for 24 page chunks, exactly matching the single certified blank skip. Both SQLite databases reported integrity_check=ok and no foreign-key violations. A live NVIDIA-vector query returned grounded vendor-catalog evidence, including the page-6 native fallback citation. Final endpoint checks

returned HTTP 200.

Quality signal

Rejecting 41 of 73 validation candidates and the only replay candidate is evidence that the source guard is active. High storage count is not the optimization target; source-grounded precision is.

CAPACITY INTERPRETATION

12. Throughput and Capacity Planning

The validation batch observed 7.90 seconds per counted page, 7.6 pages per minute, or roughly 456 pages per hour. Because failed-document time is included while failed pages are excluded from the denominator, this is a planning observation—not a clean page-only benchmark, completion claim, or service-level objective.

Scope	Pages	Arithmetic single-worker baseline	Operational planning range
Vendor catalogs	6,069	13.3 hours	16.6–20 hours
Reference library	11,768	25.8 hours	Pilot separately before commitment
Combined	17,837	39.1 hours	49–59 hours before measured concurrency gains

12.1 Bottlenecks to measure

- Nemotron Parse latency and queue depth on image-heavy/table pages.
- Structured JSON completion length, regeneration rate, and source-guard rejection distribution.
- Embedding batch latency and SQLite vector serialization size.
- CPU rendering/parse time, data-volume write pressure, checkpoint hit rate, and merge cost.
- Aggregate pages/hour and p95 stage latency as worker count moves from one to two, then conditionally to three.

RISK GOVERNANCE

13. Limitations and Risk Register

Risk / limitation	Impact	Current control	Required next control
Runner is sequential; worker settings are unused	No automatic scale-out	Bounded runs and known capacity baseline	External manifest-assigned shard workers
No manifest/page-range campaign ledger	Cannot prove full coverage or safe retry	Inventory files and unique bounded runs	Immutable campaign manifest + stable work-unit state
Whole-document evidence replacement	Page-window jobs can erase sibling evidence	Do not shard one document today	Shard-local DBs + deterministic merge
OCR endpoint not deployed	Some scan/identifier cases depend on VLM only	Fail closed; OCR hook exists	Deploy and benchmark a dedicated NVIDIA OCR verifier

Risk / limitation	Impact	Current control	Required next control
Exact cosine full scan over JSON vectors	Query latency grows linearly	Acceptable for bounded KB	sqlite-vec or external vector index; optional reranker
Embedding dimension/model not enforced	Mixed vectors may silently skip or compare incorrectly	Batch-consistency checks	Release-wide model and exact 4,096-D invariant
Citation list not schema-required for every chunk	Direct product chunks could lack evidence	Pipeline-created page chunks always cite	Require ≥ 1 citation for canonical chunks
Render profile absent from fingerprint	Changed page rendering may reuse stale VLM checkpoint	Source PDF hash	Fingerprint DPI/profile and prompt version explicitly
No general stored-response replay command	Investigation may repeat LLM/embedding work	Retain raw artifacts; rerun exact doc in new directory	Replay tool that consumes stored provider responses
Field-to-excerpt semantic proof is partial	A cited element may not fully entail an emitted value outside guarded identity cases	Unknown/cross-document evidence fails; manufacturer/row checks are strict	Exact span/value checks for identity and specification fields
JSONL and SQLite are not one transaction	Crash timing can duplicate an event	Written flags suppress ordinary retries	SQLite outbox or deterministic release-time JSONL generation
Reprocessing lacks source-scope tombstones	Older products can survive a parse that emits fewer products	Owner-scoped product and chunk replacement	Versioned source reconciliation and supersession policy
Product IDs contain model-output ordinal	Reordering equivalent products changes identifiers	Canonical JSON participates in deterministic ID	Prefer stable source identity keys when available
Model revision is not in batch identity	A model change may reuse a logical batch ID when parsed data matches	Pipeline/schema hashes are included	Add stable configuration and model-revision hash
CLI mixes raw and canonical RAG tiers	Users may conflate evidence with validated product data	Chunk kind retained; API can filter	Expose tier filter and explicit tier labeling/ranking

Primary production risk

The principal blocker is correctness of campaign partitioning and merge—not GPU capacity. The server has ample compute, but uncontrolled parallelism could create gaps, overlaps, overwritten evidence, or a release that cannot prove page coverage.

PRODUCTION ROADMAP

14. Full-Corpus Scale Plan

14.1 Immutable campaign manifests

Create separate canonical manifests for the vendor corpus and reference library. Each document entry must include corpus-relative path, SHA-256, byte size, and total pages. Canonically hash the manifest and copy that identity into every unit status, attempt summary, and merged release. Reject escaped paths, symlinks, altered hashes, altered page counts, or ambiguous duplicate-content policy before inference.

14.2 Stable page-window work units

Split PDFs into nonoverlapping one-based inclusive windows no larger than 50 pages. Persist identities such as sha-prefix-p000001-000050; never regenerate assignments during retry. Each unit writes to an isolated attempt directory and a stable checkpoint directory. Completion markers record source/campaign/config/prompt/model identities, page range, attempt, timestamps, metrics, and output hashes.

14.3 External workers and measured promotion

Campaign	50-page work units	Initial shard layout	Worker policy
Vendor	206	~24 shards averaging ~253 pages	Start with 2 workers; promote to 3 only after a stable concurrency trial
Reference library	302	~48 similar shards after a profile-specific pilot	Tune routing/quality separately; do not inherit vendor assumptions blindly

14.4 Deterministic merge and release gates

- Accept only completed/review work units whose campaign, source, pipeline, prompt, configuration, and model identities match the release.
- Create a new temporary release; import catalog rows in foreign-key order and vectors without re-embedding. Identical duplicate keys may deduplicate; conflicting payloads fail closed.
- Require every manifest page exactly once with no gaps/overlaps, no pending/running/unresolved failed units, and separate review counts.
- Require SQLite integrity_check=ok, an empty foreign_key_check, reconciled document/product/chunk/citation counts, one embedding model/dimension, release summary, and checksums.
- Publish atomically only after all gates pass; keep shard/attempt artifacts as the long-term audit source.

14.5 Delivery sequence

Phase	Objective	Core work	Exit gate
Phase 1	Campaign correctness	Manifest builder, page windows, unit ledger, isolated outputs, deterministic merge, release verifier	All new unit/merge tests pass; synthetic gaps/conflicts fail closed
Phase 2	Vendor pilot	Two external workers over representative short/long/table/scan documents	Improved aggregate throughput; no queue/OOM/JSON regression; citation QA passes
Phase 3	Vendor campaign	206 work units, ~24 shards, monitored execution and merge	6,069 pages exactly once; zero unresolved failures; published verified release
Phase 4	Library pilot/campaign	Separate extraction schema/quality profile followed by ~302 units	11,768 pages exactly once under library-specific acceptance policy
Phase 5	Combined retrieval release	Conflict-checked merge, vector-index upgrade, retrieval evaluation	Unified citation-bearing KB with release manifest and model/dimension invariant

IMPLEMENTATION GUIDE

15. Key Files and Operator Entry Points

File	Responsibility
src/industrial_catalog/routing.py	Native-quality/layout signals and deterministic route selection
src/industrial_catalog/extraction.py	PDF source, page inventory, Parse/OCR orchestration, normalization, blank/fallback policy, checkpoints
src/industrial_catalog/nvidia_clients.py	OpenAI-compatible NVIDIA requests, retries, content candidates, Parse/OCR/guided JSON clients
src/industrial_catalog/validation.py	Canonical evidence hydration, field/product validation, quality issues
src/industrial_catalog/storage.py	Catalog schema, ingestion audit, accepted/rejected persistence
src/industrial_catalog/knowledge_base.py	Chunk/citation models, embeddings, deterministic indexing, text and cosine retrieval
src/industrial_catalog/runner.py	End-to-end bounded execution and artifact lifecycle
configs/pipeline.example.yaml	Deployed endpoints, GPU/service, routing, parse, structured, and RAG defaults
scripts/start_model_services.sh	Safe model launch/adoption with GPU/model/port checks
scripts/run_benchmark.sh	Bounded validation wrapper and output isolation
scripts/monitor_pipeline.py	Endpoint/GPU/data/run status monitoring
docs/operations-and-scale-plan.md	Transfer, operations, replay, scale constraints, and campaign design

15.1 Recommended immediate work package

1. Implement and test the immutable manifest, stable page-window work-unit, and attempt-directory formats.
2. Refactor page-evidence persistence so independent windows cannot replace one another; merge into a new release database.
3. Enforce embedding model/dimension and nonempty citation invariants at index and query time.
4. Run a two-worker vendor pilot with page-level throughput and p95 stage latency instrumentation.
5. Perform human citation QA on stratified native, table, scan, fallback, blank, rejected, and accepted samples before full vendor execution.

CONCLUSION

16. Final Assessment

The implemented pipeline is architecturally sound for an evidence-first industrial catalog system. It uses the NVIDIA server where GPU inference adds the most value, keeps deterministic controls around every generative stage, and preserves enough lineage to explain both accepted and rejected outcomes. The hardened replay demonstrates the intended safety behavior: semantic VLM failure did not erase usable native evidence, a genuinely blank page did not waste inference, and an unsupported product did not enter the catalog.

The remaining work is production campaign engineering. The server's eight A100 GPUs, memory, CPUs, and storage are sufficient to support measured parallelism; however, page coverage, work-unit identity, idempotent merge, and release invariants must be solved before throughput is increased. Once those controls are implemented, the vendor corpus is the correct first full campaign, followed by a separately profiled reference-library campaign and a conflict-checked combined retrieval release.

Bottom line

Proceed to the manifest/sharding/merge milestone. Treat the current results as validated architecture and bounded operational proof—not yet as a completed industrial knowledge base.

APPENDIX A

Appendix A. Canonical Product Shape (Conceptual)

```
{
  "manufacturer": {"value": "...", "evidence": ["element_id"]},
  "part_name": {"value": "...", "evidence": ["element_id"]},
  "part_number": {"value": "...", "evidence": ["element_id"]},
  "series": {"value": "...", "evidence": ["element_id"]},
  "specifications": [
    {"name": "...", "value": "...", "unit": "...", "evidence": ["element_id"]}
  ],
  "operating_conditions": [{"name": "...", "value": "...", "evidence": ["element_id"]}],
  "other_details": [{"name": "...", "value": "...", "evidence": ["element_id"]}
]
```

The concrete Pydantic schema is the source of truth. This conceptual shape shows that normalized facts carry evidence references; the validator hydrates those references back to canonical page elements before persistence.

APPENDIX B

Appendix B. Completion Definitions

Term	Definition
Validation batch	A bounded selection that traverses extraction, structured parsing, validation, storage, embedding, and monitoring. It may include review/failure and is not corpus completion.
Completed campaign	Every page in one immutable corpus manifest has a terminal completed/review unit, no unresolved failure, and a verified merged release.
Completed combined corpus	Both vendor and library campaigns independently pass their gates, followed by a conflict-checked combined retrieval release.

APPENDIX C

Appendix C. Authoritative Model References

NVIDIA Nemotron Parse v1.2 — [official NVIDIA model card](#)

Document parsing capabilities, required task prompt, vLLM deployment, and A100 support

Llama 3.1 Nemotron Nano 8B v1 — [official NVIDIA model card](#)

Model architecture, long-context behavior, reasoning/chat/RAG/tool-use scope

Nemotron 3 Embed 8B BF16 — [official NVIDIA model card](#)

Retrieval usage, query/passage prefixes, dimensions, sequence length, and deployment

Implementation claims and measured results in this report are derived from the local project source, its operational runbook, and the verified server execution artifacts summarized there. Model capability descriptions are limited to the official NVIDIA model cards above.

APPENDIX D

Appendix D. Glossary

Term	Meaning in this report
VLM	Vision-language model that interprets rendered document pages
LLM	Language model used here for schema-constrained product parsing
RAG	Retrieval-augmented generation; here, a citation-bearing searchable knowledge base
NIM-style endpoint	OpenAI-compatible locally hosted NVIDIA model service used by the clients
Source guard	Deterministic rule layer that rejects manufacturer, cardinality, identifier, or evidence violations
Evidence hydration	Replacement of compact element references with canonical extraction evidence
Review	Completed document with accepted data plus quality warnings; not an execution failure
Fail closed	Stop or reject when evidence is insufficient instead of guessing or silently accepting